

AI SCIENTIST

FASE 2 | AULA 01 -

BANCO DE DADOS RELACIONAIS

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3

HANDS ON 4

SAIBA MAIS 5

MERCADO, CASES E TENDÊNCIAS 19

O QUE VOCÊ VIU NESTA AULA? 20

REFERÊNCIAS 21

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

O QUE VEM POR AÍ?

Sabemos que, no mundo da Ciência de Dados, ferramentas como Python, Numpy e Pandas são essenciais, mas a verdade é que o “ouro” das empresas está em bancos SQL.

Nesta disciplina, começaremos pelo alicerce. Para um(a) Cientista de Dados, entender Banco de Dados Relacionais (RDBMS) é a diferença entre apenas acessar dados e compreender a lógica do negócio por trás deles.

Vamos desmistificar como os dados “conversam” entre si. Esqueçam por um momento os arquivos “.csv” soltos. Aqui, entraremos no mundo onde a estrutura garante a qualidade. Vamos entender que uma tabela de Vendas não vive sozinha, ela precisa se conectar à tabela de Clientes, Produtos e Lojas para contar uma história completa.

Vamos dominar a linguagem universal dos dados?

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

HANDS ON

Nessa aula, abordaremos conceitos fundamentais de forma prática. Vamos aprender a traduzir objetos do mundo real para tabelas e colunas, entender o que permite cruzar informações entre as tabelas de forma precisa, sem duplicidade ou ambiguidade, organizar os dados para reduzir redundâncias e interpretar diagramas para não “se perder” no meio de milhares de tabelas. O objetivo aqui é garantir que você saiba onde os dados estão e como eles se relacionam antes mesmo de escreverem a primeira linha de código.

Afinal, um bom modelo de Machine Learning começa com uma base de dados bem compreendida. Vamos nessa!

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

SAIBA MAIS

Desenvolvido por EF Codd, da IBM, na década de 1970, o modelo de banco de dados relacional permite que qualquer tabela seja relacionada a outra tabela usando um atributo comum. Em vez de usar estruturas hierárquicas para organizar dados, Codd propôs uma mudança para o uso de um modelo de dados em que eles são armazenados, acessados e relacionados em tabelas sem reorganizar as tabelas que os contêm. Para Cientistas de Dados, entender este modelo é crucial para garantir que a extração de dados (o "E" do ETL) seja feita de forma correta e eficiente.

No MySQL, assim como em outros RDBMS (Relational Database Management Systems), podemos pensar no banco de dados relacional como uma coleção de arquivos de planilha que ajuda empresas a organizar, gerenciar e relacionar dados. No modelo de banco de dados relacional, cada "planilha" é uma tabela que armazena informações, representadas por colunas (atributos) e linhas (registros ou tuplas).

Conceitos chave:

- **Entidade:** uma representação de um objeto do mundo real (ex.: Cliente, Produto). No banco, geralmente se torna uma Tabela.
- **Atributo:** características da entidade (ex.: Nome do Cliente, Preço). Torna-se uma Coluna.
- **Domínio:** conjunto de valores permitidos para um atributo (definido pelos Data Types).

A fonte dessas informações é o Google Cloud.

ARQUITETURA E STORAGE ENGINES

Diferente de alguns bancos de dados que possuem um motor único, o MySQL opera com uma arquitetura de Storage Engines (motores de armazenamento) plugáveis. Isso permite que o banco seja otimizado para diferentes tipos de carga de trabalho.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Compreender a arquitetura interna do MySQL transforma a forma como você escreve queries, projeta schemas e otimiza o desempenho. O servidor é organizado em camadas distintas, cada uma com responsabilidades especializadas.

Camada de conexão: lida com autenticação, autorização e protocolos de segurança. Gerencia o pool de conexões e a alocação de threads.

Camada SQL (O cérebro): o Parser valida a sintaxe, o Optimizer seleciona os planos de execução e o Query Cache armazena os resultados acessados com frequência.

Camada de Storage Engine (O músculo): onde os dados residem fisicamente. InnoDB, MyISAM e outros engines gerenciam o I/O de disco real e as estruturas de memória.

INNODB (O PADRÃO MODERNO)

Para a maioria das aplicações de Ciência de Dados e sistemas transacionais modernos, o **InnoDB** é o motor padrão e o mais recomendado.

InnoDB tornou-se o storage engine padrão porque oferece confiabilidade e performance de nível empresarial. Compreender seus mecanismos é essencial para projetar data pipelines que escalam.

Por que usar? Suporta transações (ACID), chaves estrangeiras (integridade referencial) e row-level locking (bloqueio a nível de linha, permitindo alta concorrência).

Contexto DS: garante que os dados que você está extraindo para o seu DataFrame do Pandas não estão corrompidos ou incompletos.

O InnoDB domina por estes motivos:

- A conformidade total com ACID garante a integridade dos dados em todas as transações;
- O bloqueio em nível de linha permite alta concorrência sem gargalos em nível de tabela;

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

- O Buffer Pool armazena inteligentemente em cache dados frequentemente acessados na RAM;
- Mecanismos de recuperação de falhas evitam perda de dados.

O Buffer Pool é a arma secreta do InnoDB para velocidade. Ao manter páginas frequentemente acessadas na memória, ele reduz drasticamente o I/O de disco — o principal gargalo no desempenho de bancos de dados. Para workloads de ML que leem milhões de linhas, essa diferença é mensurável em minutos, não segundos.

INTEGRIDADE E CHAVES (KEYS)

A "cola" que mantém o modelo relacional coeso são as chaves. Elas impedem a duplicidade e garantem conexões válidas.

Primary Key (PK): identificador único de um registro em uma tabela.

Boas práticas: deve ser imutável e nunca nula (NOT NULL). Geralmente utiliza-se um INT AUTO_INCREMENT ou um UUID ((Universally Unique Identifier) é um identificador alfanumérico de 128 bits).

Impacto na performance: o MySQL (InnoDB) armazena os dados fisicamente organizados pela Primary Key (Clustered Index). Buscas pela PK são as mais rápidas possíveis.

Foreign Key (FK): um campo que aponta para a PK de outra tabela.

Função: garante a **Integridade Referencial**. O banco impede que você insira uma venda para um cliente que não existe na tabela de clientes.

Nota: alguns ambientes de Data Warehouse removem as FKs para acelerar a carga de dados, mas em bancos de produção (OLTP), elas são essenciais.

NORMALIZAÇÃO DE DADOS

A normalização é o processo de organizar os dados para reduzir a redundância e melhorar a integridade, é essencial no design de banco de dados relacionais para evitar anomalias de atualização.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

- **1ª Forma Normal (1NF):** atomicidade. Cada célula da tabela deve conter um único valor (nada de listas separadas por vírgula em uma coluna).
- **2ª Forma Normal (2NF):** a tabela deve estar em **1NF** e todos os atributos não-chave devem depender **inteiramente** de **toda** a chave primária composta. Se a chave primária for composta (feita de duas ou mais colunas), nenhum atributo não-chave pode depender de apenas uma parte dessa chave.
- **3ª Forma Normal (3NF):** a tabela deve estar em **2NF** e não deve haver dependência transitiva (atributos não-chave dependendo de outros atributos não-chave).

O dilema da pessoa cientista de dados: enquanto a normalização é perfeita para **escrever** dados (OLTP) sem erros, ela pode tornar a **leitura** (OLAP) lenta, pois exige muitos JOINS. Em projetos de Analytics, é comum trabalharmos com dados **desnormalizados** (tabelões ou Star Schemas) para facilitar a análise.

O esquema em estrela (Star schemas) é uma abordagem de modelagem madura amplamente adotada por data warehouses relacionais. Ele requer que os modeladores classifiquem suas tabelas de modelo como dimensão ou fato.

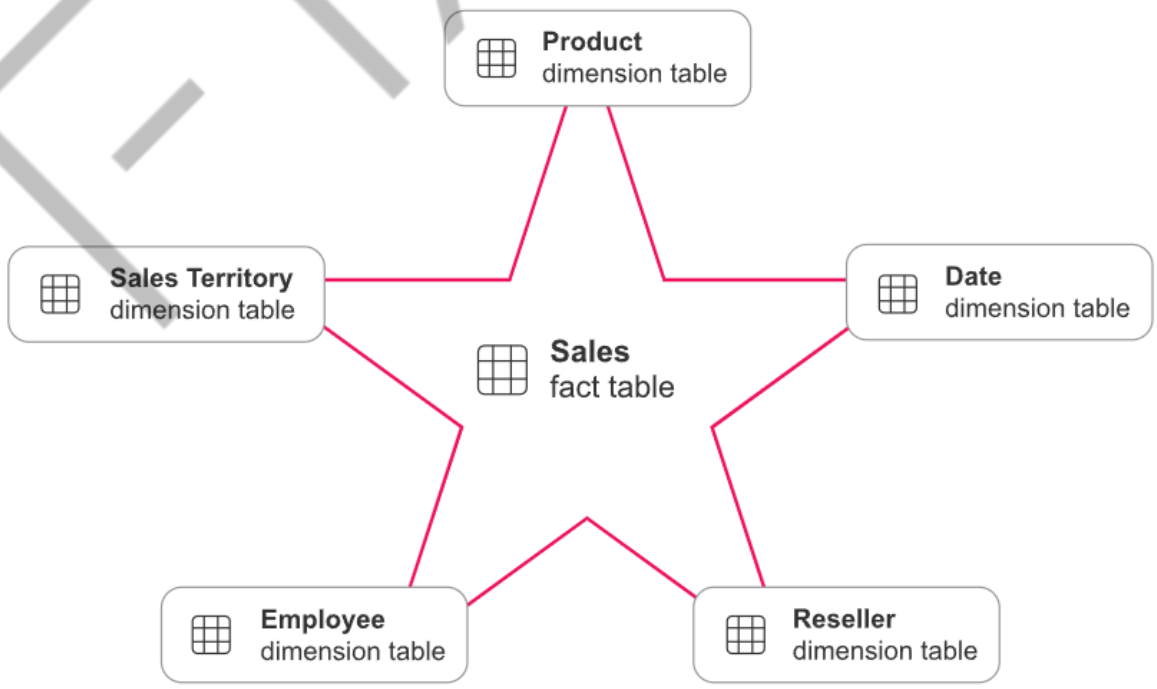


Figura 1 – Star Schema

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Fonte: Microsoft (s.d.)

PROPRIEDADES ACID

Para confiar nos dados, o MySQL adere às propriedades ACID. Isso é vital quando estamos capturando dados em tempo real ou construindo pipelines de dados críticos. As propriedades ACID são o conjunto fundamental de regras que os sistemas de gerenciamento de banco de dados (SGBDs) relacionais, como o MySQL (usando o mecanismo InnoDB), devem seguir para garantir que as transações de dados sejam processadas de forma **confiável**.

É a garantia de que, mesmo em ambientes de alta concorrência ou falhas de sistema, seus dados permanecerão precisos e íntegros.

A - Atomicidade: a transação é "tudo ou nada". Se uma parte falhar, tudo é desfeito (rollback), ela deve ser tratada como uma **única unidade de trabalho indivisível**. Ou todas as operações dentro dela são concluídas com sucesso, ou nenhuma delas é concluída.

O que acontece: se qualquer instrução SQL dentro da transação falhar (por exemplo, uma violação de chave primária, falta de espaço em disco ou erro de lógica), o SGBD realiza um **rollback**, desfazendo todas as modificações que já haviam sido feitas por essa transação. Isso impede que o banco de dados fique em um estado parcialmente atualizado.

Exemplo prático (Transferência bancária):

1: DEBITAR $100 da Conta A.

2: CREDITAR $100 na Conta B.

Se o débito funcionar (1), mas o crédito falhar (2), a Atomicidade garante que o débito também seja desfeito. O dinheiro não pode simplesmente "desaparecer" no meio do caminho.

C - Consistência: a transação leva o banco de um estado válido para outro estado válido (respeitando as Constraints e Foreign Keys). O banco de dados nunca deve violar suas regras predefinidas (esquema, constraints, chaves estrangeiras, regras de validação).

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

O que acontece: se uma transação tentar fazer uma alteração que viole uma regra (por exemplo, tentar inserir um valor NULL em uma coluna NOT NULL, ou tentar inserir um ID_Departamento que não existe na tabela Departamentos), a transação será abortada (rollback) para garantir a integridade dos dados.

Exemplo prático (Integridade referencial):

1. Você tem uma tabela Pedidos com uma Chave Estrangeira (FK) referenciando a tabela Clientes.
2. Se uma transação tentar inserir um pedido com um ID_Cliente que não existe na tabela Clientes, a Consistência (aplicada pela FK) rejeita a transação, mantendo a integridade referencial do banco.

I - Isolamento: uma transação em andamento não deve interferir em outra. O MySQL oferece níveis de isolamento (ex.: REPEATABLE READ, padrão do InnoDB). A execução simultânea de várias transações deve produzir o mesmo resultado que a sua execução serial (uma após a outra). Em outras palavras, uma transação em andamento não pode ser afetada ou visível para outra transação em andamento.

O que acontece: o MySQL (InnoDB) gerencia isso usando locks (bloqueios) e diferentes níveis de Isolamento, como:

- READ UNCOMMITTED**:** menor isolamento; pode ler dados não confirmados (dirty reads).
- READ COMMITTED**:** impede dirty reads.
- REPEATABLE READ **(Padrão do InnoDB):** garante que, dentro de uma transação, leituras repetidas retornem o mesmo dado, impedindo non-repeatable reads.
- SERIALIZABLE**:** maior isolamento; impede todos os problemas, mas impacta mais o desempenho.

Exemplo prático (Leitura concorrente):

1. Transação A inicia, lê um valor X.
2. Transação B inicia, altera o valor X para Y e faz **COMMIT**.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

3. Transação A lê o valor X novamente.

No nível **REPEATABLE READ**, a Transação A ainda veria o valor original X, agindo como se a Transação B nunca tivesse ocorrido até que A termine. Isso garante a consistência lógica de A.

D - Durabilidade: uma vez confirmada (COMMIT), a mudança é permanente, e devem sobreviver a qualquer falha subsequente do sistema (queda de energia, falha de hardware etc.), garantido pelos logs de transação/Redo Log.

O que acontece: o MySQL garante a Durabilidade escrevendo as informações da transação e das alterações nos logs de transação (Redo Log e Binary Log) antes de confirmar que a transação foi concluída com sucesso para o usuário.

Se o sistema cair logo após o COMMIT, na reinicialização (recuperação), o MySQL usa esses logs para refazer quaisquer alterações que não tenham sido totalmente escritas nos arquivos de dados principais no disco.

Exemplo prático (Queda de energia):

1) Você faz uma transação que insere novos dados.
2) Você executa COMMIT. O MySQL grava a operação no Redo Log e confirma.
3) O servidor tem uma queda de energia imediata.
4) Quando o servidor é reiniciado, o MySQL verifica o Redo Log, encontra a transação confirmada, e garante que ela seja escrita nos arquivos de dados, restaurando a Durabilidade.

O QUE ACONTECE DEBAIXO DOS PANOS

Cada query SQL segue um caminho de execução preciso através da arquitetura do MySQL. Compreender esse pipeline ajuda você a escrever queries que aproveitam, em vez de lutar contra o otimizador.

Conexão e Autenticação: o MySQL valida as credenciais e aloca um thread de conexão do pool.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Analisador e pré-processador: a validação da sintaxe ocorre primeiro. O analisador constrói uma árvore de análise e verifica a existência de tabelas/colunas e permissões.

Otimizador de Query: o cérebro do MySQL analisa índices disponíveis, estatísticas de tabela e possíveis ordens de junção para selecionar o plano de execução ideal

Motor de Execução: coordena com o motor de armazenamento para recuperar dados, aplicando filtros e transformações conforme especificado

I/O do Motor de Armazenamento: o InnoDB verifica primeiro o Buffer Pool. Se os dados não estiverem em cache, ele os lê do disco para a memória

Montagem do Resultado: os resultados são empacotados e transmitidos de volta para o seu Python DataFrame ou aplicação

TIPOS DE DADOS RELEVANTES PARA ANÁLISE

Além dos clássicos INT, DECIMAL e VARCHAR, o MySQL moderno oferece suporte robusto para dados que a pessoa Cientista de Dados encontra frequentemente:

DATETIME / TIMESTAMP: estes dois tipos são cruciais, mas é importante entender suas diferenças sutis no MySQL

Aplicação em análise

O verdadeiro poder está nas **funções nativas do MySQL**. Elas permitem a Engenharia de Recursos (Feature Engineering) diretamente no banco, antes de passar para o modelo:

- **Extração:** funções como EXTRACT(HOUR FROM datetime_column), YEAR(), MONTH(), DAYOFWEEK(), e WEEK() permitem desagregar a data/hora em componentes úteis para identificar sazonalidade e padrões.
- **Cálculo de Duração:** DATEDIFF(data1, data2) e TIMESTAMPDIFF(unit, data1, data2) são vitais para calcular a latência entre eventos (ex.: tempo que um cliente leva para concluir uma compra após o login).

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

- **Janelamento (Windowing):** as funções de janela (disponíveis em versões recentes) permitem calcular médias móveis, somas acumuladas ou a diferença do valor anterior/próximo em uma série temporal sem precisar exportar os dados primeiro.

JSON

O MySQL suporta nativamente o tipo de dados JSON, permitindo armazenar documentos semiestruturados dentro de um banco relacional e consultá-los via SQL. Isso é extremamente útil para lidar com logs de aplicações ou dados de APIs.

Por que é poderoso para o(a) Cientista de Dados?

Esquema flexível: você pode armazenar dados com estruturas variáveis, como payloads de APIs, configurações de usuários, ou logs de eventos que não possuem um esquema fixo.

Consultas nativa e otimizada: em vez de armazenar o JSON como VARCHAR (o que exige parsing lento após a recuperação), o tipo JSON armazena o dado em um formato binário otimizado. Isso permite que o MySQL use funções poderosas para extrair e manipular os dados diretamente via SQL, de forma indexável e performática.

Funções-Chave (JSON Functions):

- JSON_EXTRACT(coluna, 'caminho'): extrai um valor específico de um objeto JSON.
- Ex.: JSON_EXTRACT(dados, '$.endereco.cidade')
- JSON_CONTAINS(coluna, valor, 'caminho'): v erifica se um valor está presente no documento JSON.
- **Gerando JSON:** JSON_OBJECT(), JSON_ARRAY() e JSON_MERGE_PATCH() permitem construir objetos JSON complexos a partir de dados relacionais, facilitando a exportação ou a resposta a APIs.

Ao usar as **funções JSON**, você pode filtrar e agregar informações complexas antes de levar os dados para o Python/R, reduzindo a carga de processamento na camada de aplicação.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

A seleção de tipos não se trata apenas de correção — é sobre custo, velocidade e escalabilidade. Cada byte importa quando você está processando bilhões de linhas em pipelines de ML.

INTRODUÇÃO À PERFORMANCE E ÍNDICES

Os índices são estruturas especiais de pesquisa que permitem ao banco de dados localizar dados de forma muito mais rápida. No entanto, sua criação e manutenção não vêm sem um custo, gerando um trade-off entre a velocidade de leitura e escrita.

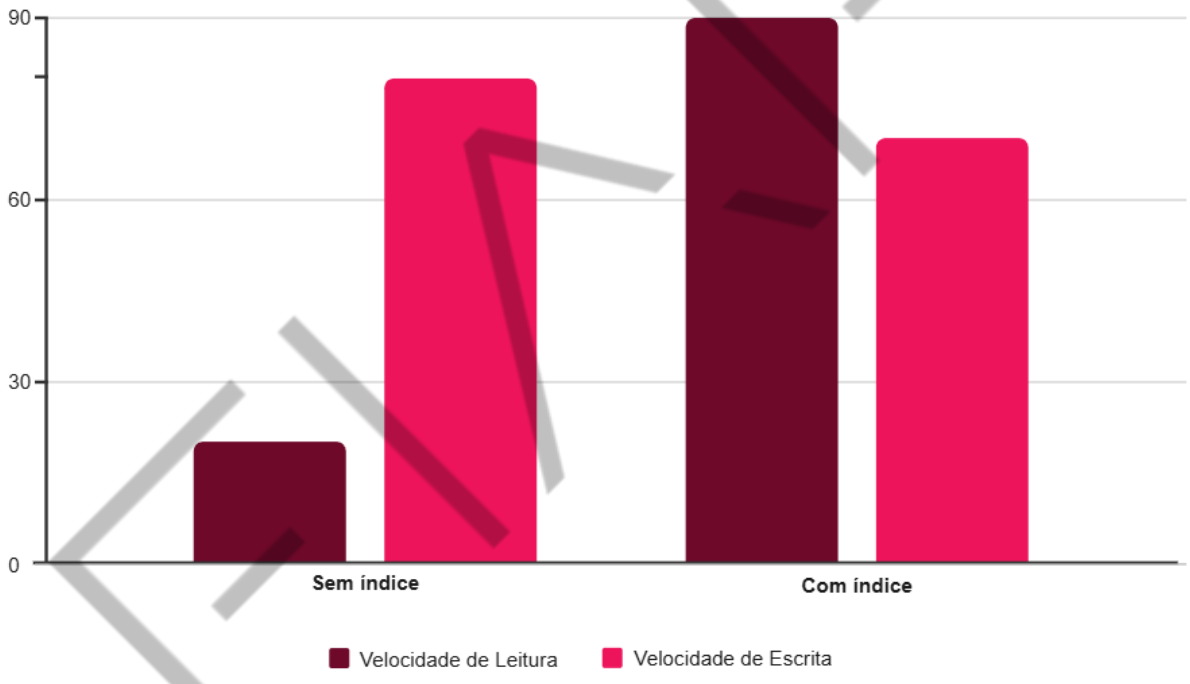


Figura 2 - Índices
Fonte: elaborado pelo autor (2026)

Como o gráfico ilustra, a adição de índices otimiza drasticamente a velocidade de leitura, essencial para consultas frequentes, mas pode impactar ligeiramente a performance de escrita devido à necessidade de atualizar a estrutura do índice.

Como o MySQL encontra uma agulha no palheiro? Sem um índice, ele faz um Full Table Scan, ou seja, lê o livro inteiro para achar uma frase. O índice é como o índice remissivo de um livro: ele aponta exatamente onde o dado está. Mas atenção:

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

índices não são de graça. Eles aceleram a leitura, mas tornam a escrita um pouco mais lenta, pois o banco precisa atualizar o índice a cada novo dado. Lembrem-se: um índice em uma coluna mal tipada é como colocar um motor de Ferrari em um chassi de plástico. A performance real começa na escolha do tipo de dado.

FOCO NO ISOLAMENTO E SEUS NÍVEIS

No item **ACID**, introduzimos o conceito de Isolamento (I), que garante que transações concorrentes não interfiram umas nas outras. Na prática do MySQL (InnoDB), isso é controlado pelos **Níveis de Isolamento**.

Para o(a) Cientista de Dados, escolher (ou apenas entender) o nível de isolamento é fundamental, pois afeta a **consistência de leitura** dos dados que estão sendo extraídos.

Nível de Isolamento (MySQL)	Visão da Transação	Impacto na Leitura do(a) Cientista de Dados
READ UNCOMMITTED	Vê alterações que ainda não foram confirmadas (Dirty Reads).	**Alto risco:** se a transação for desfeita (rollback), você extraiu dados que nunca existiram.
READ COMMITTED	Vê apenas alterações que foram confirmadas (committed).	**Risco médio:** pode ver dados diferentes se repetir a leitura dentro da mesma transação (Non-repeatable Reads).
REPEATABLE READ (Padrão do InnoDB)	Garantia de que a transação verá os mesmos dados do início ao fim.	**Risco controlado:** Altamente seguro para consistência, mas pode mascarar dados novos inseridos por fora

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

		(Phantom Reads são tratados de forma especial no InnoDB).
SERIALIZABLE	Todas as transações são executadas sequencialmente.	**Menor risco, pior performance:** máxima consistência, mas impõe bloqueios que travam o banco, inviável para longas consultas em produção (OLTP).

Contexto prático: se sua query de extração for longa (1 hora) e você estiver usando o REPEATABLE READ, o dataset extraído será um snapshot do momento em que sua query começou garantindo coerência, mesmo que milhares de novas vendas tenham sido adicionadas no meio.

OLTP VS. OLAP: ENTENDENDO A ORIGEM E O DESTINO DOS DADOS

A infraestrutura de dados de uma empresa é geralmente dividida em dois grandes grupos:

OLTP (Online Transaction Processing)

Finalidade: processamento de transações (Ex.: registrar vendas, atualizar inventário, cadastrar usuários).

Características:

- **Foco: escrita** rápida, alta disponibilidade e segurança (ACID).
- **Estrutura:** altamente **normalizada** (muitas tabelas, poucos dados redundantes).
- **Consultas:** curtas e simples (Ex.: INSERT, UPDATE de uma única linha).

Onde o(a) cientista de dados atua: é a fonte primária dos dados brutos.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

OLAP (Online Analytical Processing)

Finalidade: análise de Negócios, geração de relatórios e modelagem preditiva.

Características:

- **Foco: leitura** e agregações complexas rápidas.
- **Estrutura: desnormalizada** (poucas tabelas, dados redundantes, esquemas Star ou Snowflake).
- **Consultas:** longas e complexas (GROUP BY extensos, agregação de meses/anos de dados).

Onde o(a) Cientista de Dados atua: é o **ambiente de destino** (Data Warehouse / Data Mart), onde o SQL analítico será mais frequentemente executado.

Dominar SQL e a Otimização é essencialmente aprender a transformar e transportar dados do eficiente ambiente **OLTP** (normalizado, para escrita) para o ambiente **OLAP** (desnormalizado, para leitura) da forma mais eficiente e rápida possível.

A normalização existe para proteger o banco de erros e economizar espaço. Uma alteração de preço acontece em um só lugar, e não em mil linhas. Isso é o mundo OLTP (Transacional). Mas, para nós, o excesso de normalização tem um custo: o 'imposto do JOIN'. Fazer 10 JOINs para treinar um modelo pode destruir a performance. Por isso, na análise, muitas vezes fazemos o caminho inverso — a desnormalização — criando as 'Flat Tables' no mundo OLAP (Analítico). O segredo é saber quando respeitar a regra e quando quebrá-la para ganhar velocidade no pipeline de dados.

DADOS INFLADOS = CONTA CARA! A REALIDADE DO CUSTO DA NUVEM

As instâncias de banco de dados em nuvem são cobradas com base em IOPS Input/Output Operations Per Second (Operações de Entrada e Saída por segundo), armazenamento e consumo de memória. Escolhas do tipo de dados inadequadas forçam você a provisionar instâncias maiores do que o necessário.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Quando seus dados de treinamento estão no MySQL, cada ineficiência no design do esquema se traduz diretamente em:

1. Tempos de extração de dados mais longos.
2. Custos de transferência de rede mais altos.
3. Requisitos de memória aumentados para processos ETL.
    - Iterações de treinamento de modelo de ML mais lentas.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

MERCADO, CASES E TENDÊNCIAS

Terminamos entendendo que o Banco de Dados Relacional (RDBMS) não é uma tecnologia do passado; é o **alicerce** da maioria dos sistemas de dados estruturados no mundo. Para um(a) Cientista de Dados, esse domínio é um diferencial competitivo crucial, pois a maior parte do "ouro" transacional das empresas reside em sistemas como o MySQL.

O mercado valoriza imensamente o(a) profissional que não apenas utiliza frameworks de Machine Learning em Python, mas que sabe onde os dados nascem, como são protegidos (ACID) e como são estruturados. Enquanto novas ferramentas de NoSQL (como MongoDB ou Cassandra) ganham espaço para dados não estruturados, o SQL continua sendo a língua para dados de clientes, finanças e inventário. Empresas buscam Cientistas de Dados que possam fazer a ponte entre os(as) **Engenheiros(as) de Dados** (que mantêm os sistemas OLTP) e os(as) **Analistas de Negócios** (que precisam de relatórios). O conhecimento profundo de RDBMS e o modelo OLTP/OLAP capacita a ser esse tipo de tradutor(a) técnico(a).

O RDBMS não parou no tempo. O MySQL e outros bancos relacionais estão se adaptando ativamente às necessidades modernas, o que torna o conhecimento dos fundamentos ainda mais relevante:

- **Suporte a JSON Nativo:** o MySQL não é mais apenas para dados estruturados. O suporte robusto ao tipo de dado **JSON** permite armazenar dados semiestruturados.
- **Convergência de Dados:** a tendência é de convergência. O mercado está vendo a fusão de bancos de dados que podem lidar com transações (OLTP) e análises complexas (OLAP) no mesmo lugar.
- **Cloud e Escalabilidade:** plataformas de Cloud (AWS RDS, Google Cloud SQL) oferecem o MySQL como serviço gerenciado.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nessa aula, saímos da mentalidade de planilhas e mergulhamos no coração dos **Bancos de Dados Relacionais (RDBMS)**, utilizando o **MySQL** com o motor **InnoDB** como nosso principal exemplo.

Os pontos de maior impacto que cobrimos foram:

Estrutura é Consistência: aprendemos que a base de um RDBMS é a organização em **Entidades (Tabelas)** e **Atributos (Colunas)**, definidos por seus **Tipos de Dados**.

A "cola" dos dados: entendemos a importância das **Chaves Primárias (PK)** para identificação única e, crucialmente, das **Chaves Estrangeiras (FK)** para impor a **Integridade Referencial** entre as tabelas.

Segurança (ACID): discutimos as quatro propriedades vitais que garantem a confiabilidade dos dados: **Atomicidade**, **Consistência**, **Isolamento** e **Durabilidade**. Reforçamos como os **Níveis de Isolamento** afetam a leitura dos dados em ambientes de produção.

Organização (Normalização): vimos que a **Normalização** (Ex.: 3NF) é crucial para sistemas transacionais (OLTP), mas introduzimos o trade-off de desempenho que nos fará, na análise (OLAP), pensar em desnormalização.

A principal lição é: a pessoa Cientista de Dados precisa não apenas saber usar o SQL, mas entender como a estrutura e as regras de um RDBMS protegem e definem a qualidade dos dados que alimentarão seus modelos.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

REFERÊNCIAS

MySQL. **Chapter 17 The InnoDB Storage Engine**. Disponível em: https://dev.mysql.com/doc/refman/8.4/en/innodb-storage-engine.html. Acesso em: 10 mar. 2026.

MySQL. **InnoDB and the ACID Model**. Disponível em: https://dev.mysql.com/doc/refman/8.4/en/mysql-acid.html Acesso em: 10 mar. 2026.

MySQL. **MySQL Documentation**. Disponível em: https://dev.mysql.com/doc/. Acesso em 10 mar 2026.

MySQL. **The DATE, DATETIME, and TIMESTAMP Types**. Disponível em: https://dev.mysql.com/doc/refman/8.4/en/datetime.html Acesso em: 10 mar. 2026.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

PALAVRAS-CHAVE

RDBMS, SQL, MySQL, ACID, OLTP, OLAP, ETL.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com



PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com